数据库原理

Database Principle

陶宏才，李晓航

西南交通大学

计算机与人工智能学院

第七章 故障恢复

学习目的和要求

- 事务管理概况
- 故障恢复导论
- 日志结构
- 更新事务的执行与恢复
- 消息的处理
- 失效的类型及恢复对策



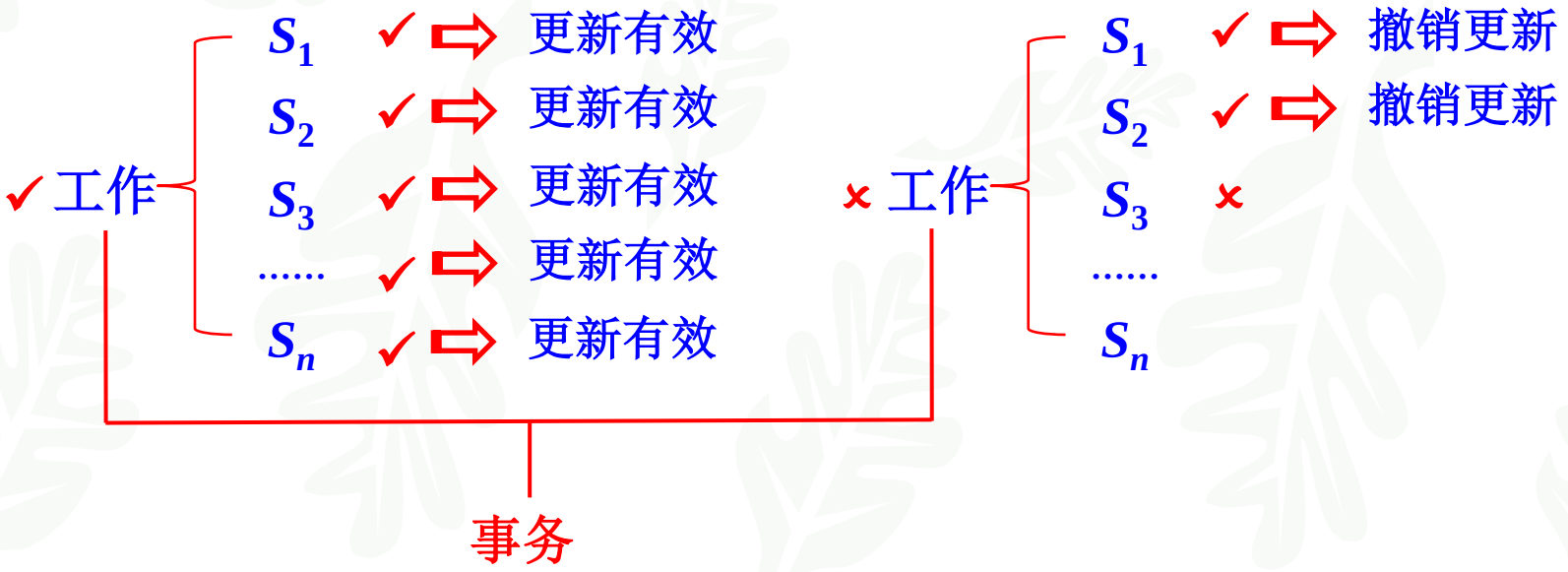
主要内容


事务管理基础
事务概念：最小执行/恢复/并发单位
控制方式：隐式、显式（BEGIN/COMMIT/ROLLBACK）
ACID准则：**原子性、一致性、隔离性、持久性**
事务管理：故障恢复+并发控制
故障恢复技术
单纯后备副本：全量/增量转储
后备+日志：前像BI(Undo)、后像AI(Redo)
多副本：镜像、复制、群集
日志核心
内容：ATL、CTL、BI文件、AI文件
规则：提交规则、先记后写(WAL)
更新事务执行
方案1：提交前AI写入DB
方案2：提交后AI写入DB
方案3：提交前后AI写入DB
消息处理
消息管理器：事务结束前暂存消息
机制：发送-确认、编号防重
故障类型与恢复
事务失效：仅Undo
系统失效：Undo+Redo，**检查点优化**
介质失效：仅Redo，需人工干预

7.1 事务管理概况

一.事务基本概念

事务概念：有时，某个工作的完成要分若干步骤。只有所有步骤都成功做完，则该项工作才完成；否则，其中任一步失败，该工作亦失败。针对此类工作特点，引入“事务”（Transaction）概念。在DBMS中，定义此类工作为事务，并保证其执行特点。



事务的执行保证：DBMS应保证事务执行特点以及数据库的数据完整性。如果事务执行成功，DBMS应保证事务中所有步骤对数据的更新有效；若事务失败，DBMS应撤销“已执行步骤”对数据的更新。

事务典型事例：银行A、B两个帐户间的转帐工作。该工作分二步来完成，Step 1: 从A帐户划出X款项；Step 2: 向B帐户划入X款项。只有这两步都完成了，该转帐工作才完成。否则，任一步失败，则该转帐工作失败。

Step 1:

```
UPDATE  Accounts
SET balance = balance – x
WHERE account_id='A‘
```

Step 2:

```
UPDATE  Accounts
SET balance = balance + x
WHERE account_id='B'
```

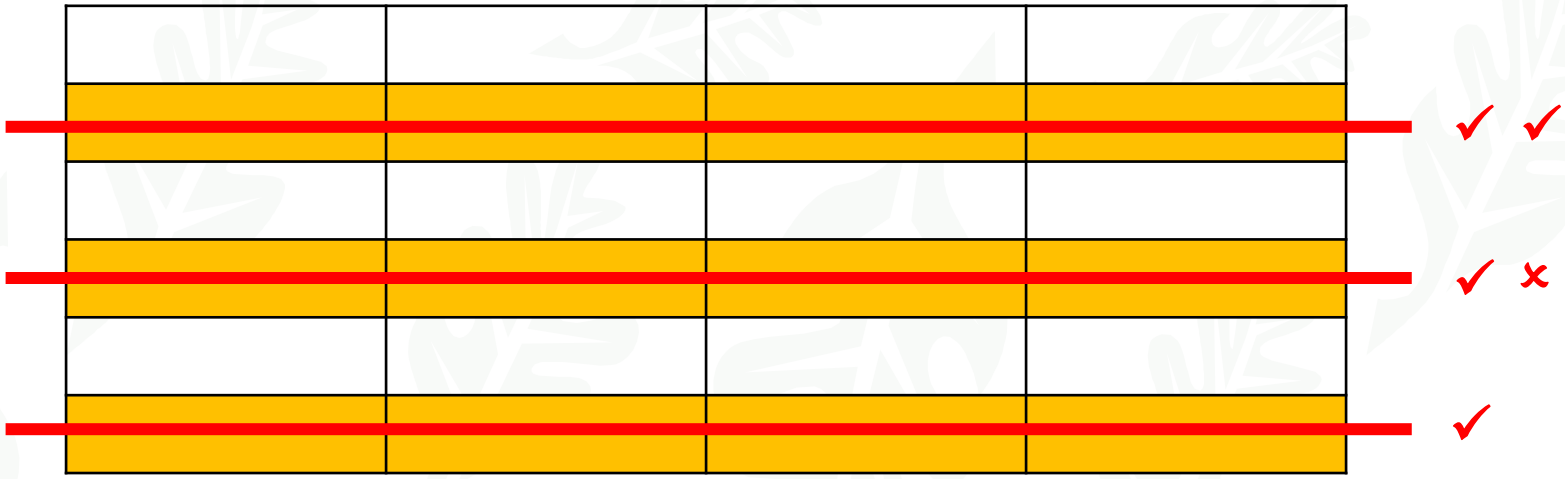
事务的组成及执行：事务是DBMS的最小执行单位/单元，由有限的数据库操作序列组成，也是最小的故障恢复单位和并发控制单位。

说明：对数据库的操作必须自成/组合为一个事务单位，以事务为最小单位执行之；在并发执行时，亦是以事务为单位进行；在事务不完整、需要恢复数据时，仍然是以事务为单位进行。

DBMS中的事务控制：

(1) 隐式的事务控制：默认情况下，DBMS一般将一个数据库操作（如一条SQL语句）当作一个事务来控制执行。

说明：事实上，有时一条SQL语句的工作也有事务特点，例如一条删除多行数据的SQL语句。



(2) 显式的事务控制：对涉及多步操作的（一般含多条SQL语句）、有事务特点的工作，则需要人为地、显式地将这些操作“界定”组合成一个事务交DBMS控制执行。

SYBASE/SQL SERVER提供的界定事务的常用语句：

① BEGIN { TRANSACTION | **TRAN** | WORK } [事务名]

② ROLLBACK { TRANSACTION | TRAN | WORK } [事务名]

③ COMMIT { TRANSACTION | TRAN | WORK } [事务名]

事务控制语句使用示例

```
BEGIN TRAN  accout_transfer
    UPDATE  Accounts
    SET balance = balance – x
    WHERE account_id='A‘
    IF @@transtate = 2                    /* 有错误出现 */
        ROLLBACK TRAN                     /* 回退事务 */

    UPDATE  Accounts
    SET balance = balance + x
    WHERE account_id='B'
    IF @@transtate = 2                    /* 有错误出现 */
        ROLLBACK TRAN
    ELSE
        COMMIT TRAN                       /* 提交事务 */
```

注：SQLServer不支持@@transtate，可使用@@TRANCOUNT或XACT_STATE()，配合begin try…end try begin catch…end catch实现事务逻辑

事务的地位：是DBMS中并发执行(Concurrent Execution)和故障恢复(Recovery from system failure)的基础。

有关概念：

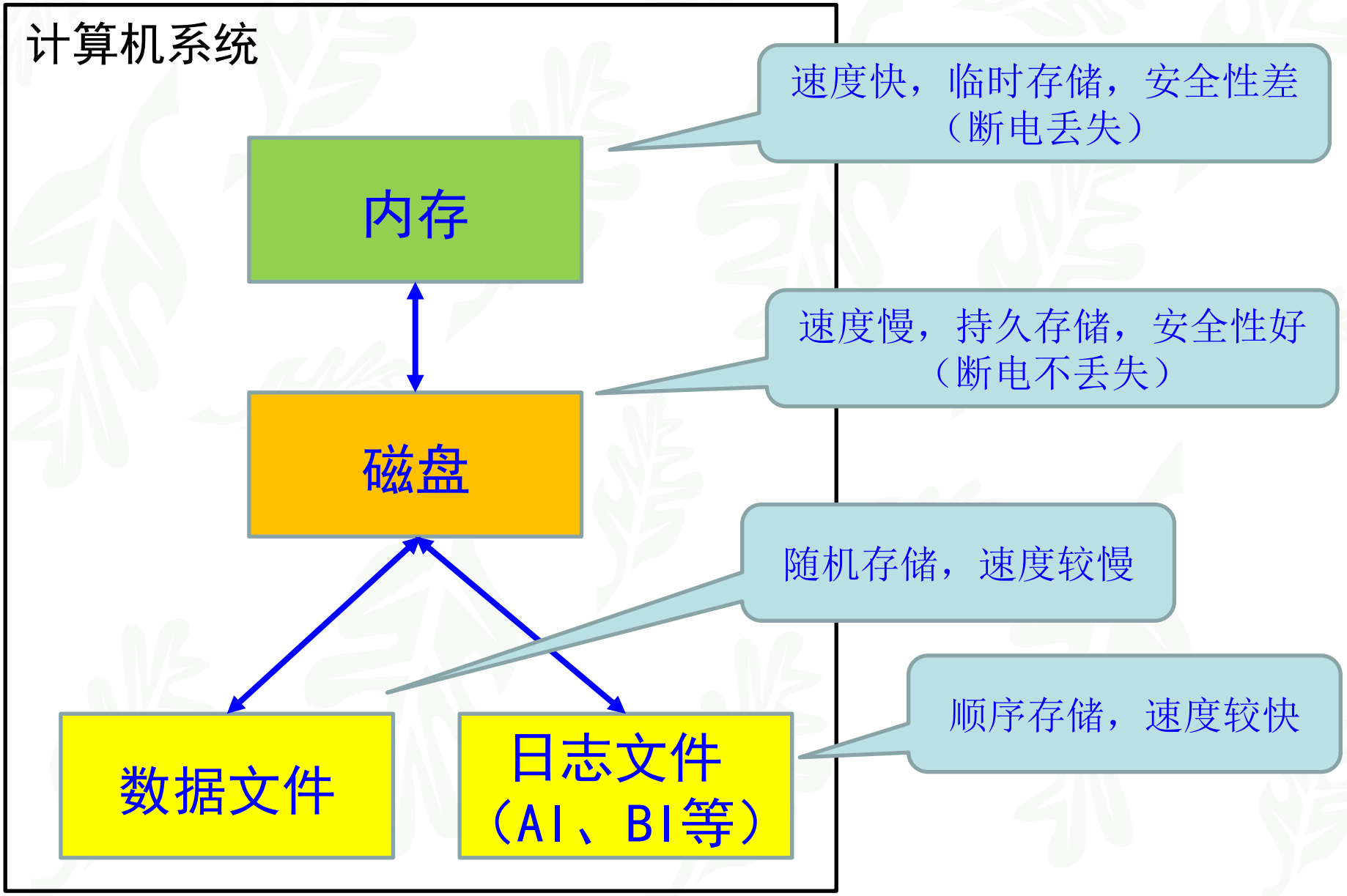
① DB对象：程序R/W信息的单位(如页、记录等)；

② 读DB对象：从磁盘读入内存缓冲区，再送程序变量中；

③ 写DB对象：修改内存缓冲区中对象副本，后写入磁盘。

④ 修改DB对象：将DB对象从磁盘读入内存，修改内存中该对象副本，然后写入磁盘。

附：计算机系统存储结构


计算机系统
内存
速度快，临时存储，安全性差
（断电丢失）
磁盘
速度慢，持久存储，安全性好
（断电不丢失）
随机存储，速度较慢
数据文件
日志文件
（AI、BI等）
顺序存储，速度较快

二. 事务的ACID准则

DBMS为保证在并发访问和故障情况下对数据的维护，要求事务有如下四个重要特征或准则(ACID)：

(1)原子性(Atomicity)：事务中的所有操作要么成功执行，要么都不执行(Nothing or All)。其保证由事务**管理器**(Transaction Manager)负责，“提交(Commit)”和“回退(Rollback)”操作是其中的关键。

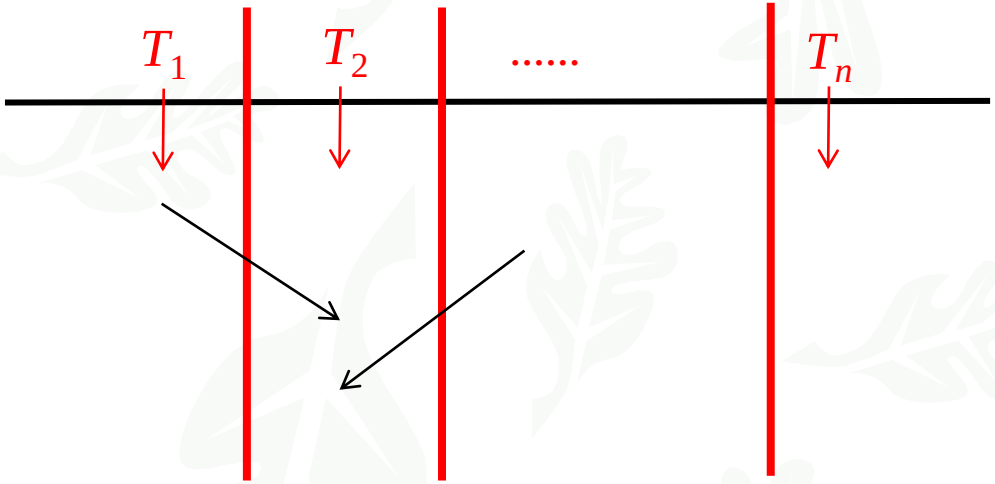
说明：

① COMMIT表明事务成功结束：告诉事务管理器事务成功完成，DB又处于一致状态，该事务的所有更新操作现可被提交或永久保留。

② ROLLBACK表明事务不成功结束：告诉事务管理器出现故障，DB可能处于不一致状态，该事务中已做的所有更新操作必须回退或撤销(Undo)。事务回退后，DB又处于一致状态。

(2)一致性(Consistency)：在无其它事务并发执行时，单独运行的事务应保证DB从一个一致状态转到另一个一致状态，但事务内勿需保证一致性。该特性的一部分需由用户负责。

(3)隔离性(Isolation)：DBMS为改善性能要交错执行几个事务的操作，但要求这些并发执行的事务，对用户而言象是单独执行一样。该特性由并发控制负责。



(4)持久性(Durability)：一旦DBMS告诉用户事务成功执行，其对数据库的影响应是持久的，即使事务导致的变化在写盘之前系统崩溃仍应如此。该特性和第一个特性由故障恢复通过日志负责。

三. 事务管理的内容

引起事务不完全的三个故障原因：

①由于出现异常，中途中止或不成功退出（事务故障）；

②可能因电源等故障，系统崩溃（系统故障）；

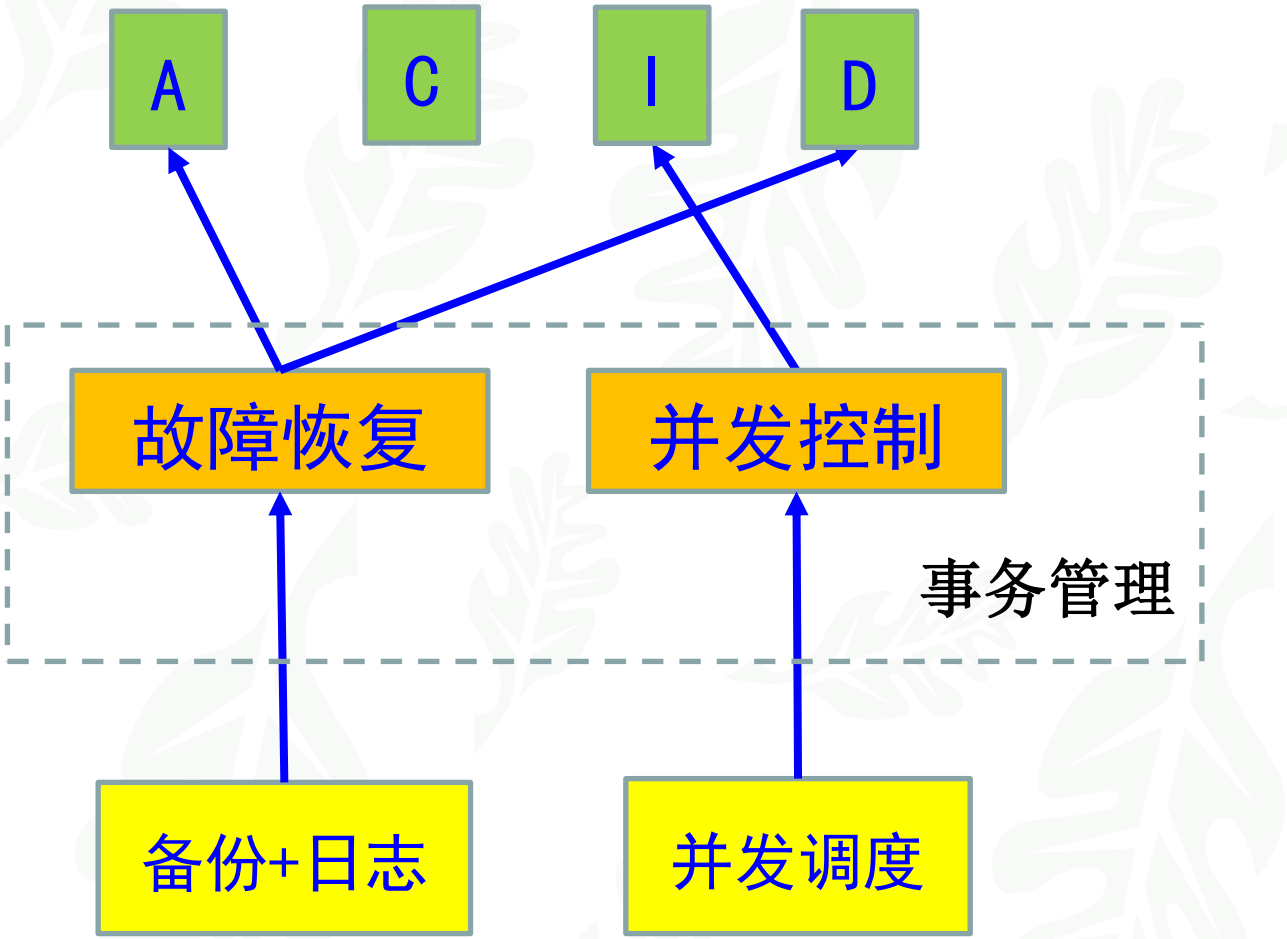
③遇到如不能访盘等异常状态而中止（介质故障）。

ACID准则的保证：不仅在系统正常如此，在系统故障时也应如此；在单个事务执行时如此，在事务并发执行时也应如此。

故障恢复（Crash Recovery）：保证事务在故障时满足持久性（和原子性）的技术；

并发控制（Concurrency Control）：保证事务在并发执行时满足隔离性的技术；

事务管理（Transaction Management）：故障恢复和并发控制的合称。


A
C
I
D
故障恢复
并发控制
事务管理
备份+日志
并发调度

7.2 故障恢复导论

故障 → 数据丢失 → 数据恢复。

备份副本（Copy）：恢复丢失数据的手段。

恢复：主要指恢复DB本身，即在故障引起DB当前状态不一致后，将DB恢复到某个正确状态或一致状态。要恢复，必冗余(Redundancy)。

一.单纯以后备副本为基础的恢复技术

概念：周期性(天、周)转储（Dump）DB到磁带/磁盘上。

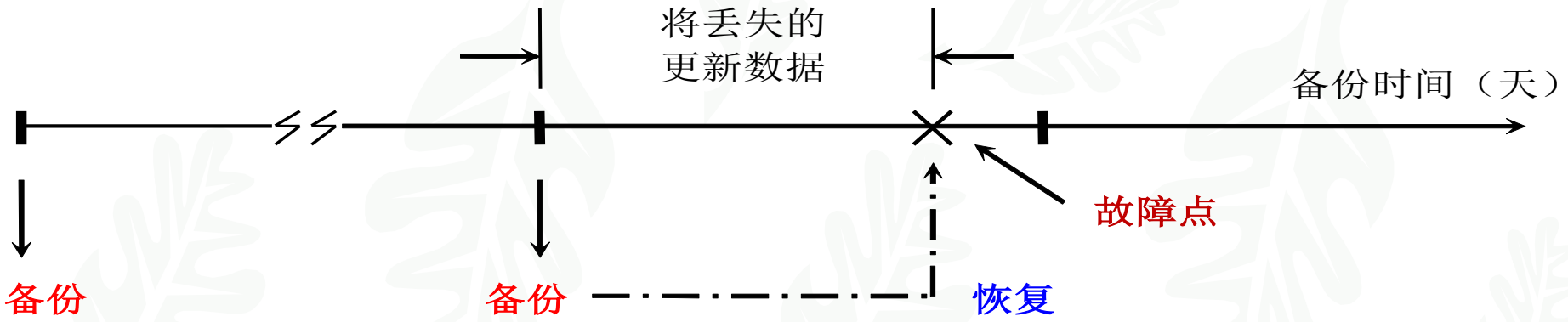


图 单纯以后备副本为基础的DB恢复

备份时间：一般在夜间、周末；

恢复方法：DB失效时，取最近的后备副本来恢复DB。

改进方法：增量转储(Incremental Dumping, ID)，只转储修改过的物理块，减少转储时间和数据丢失。

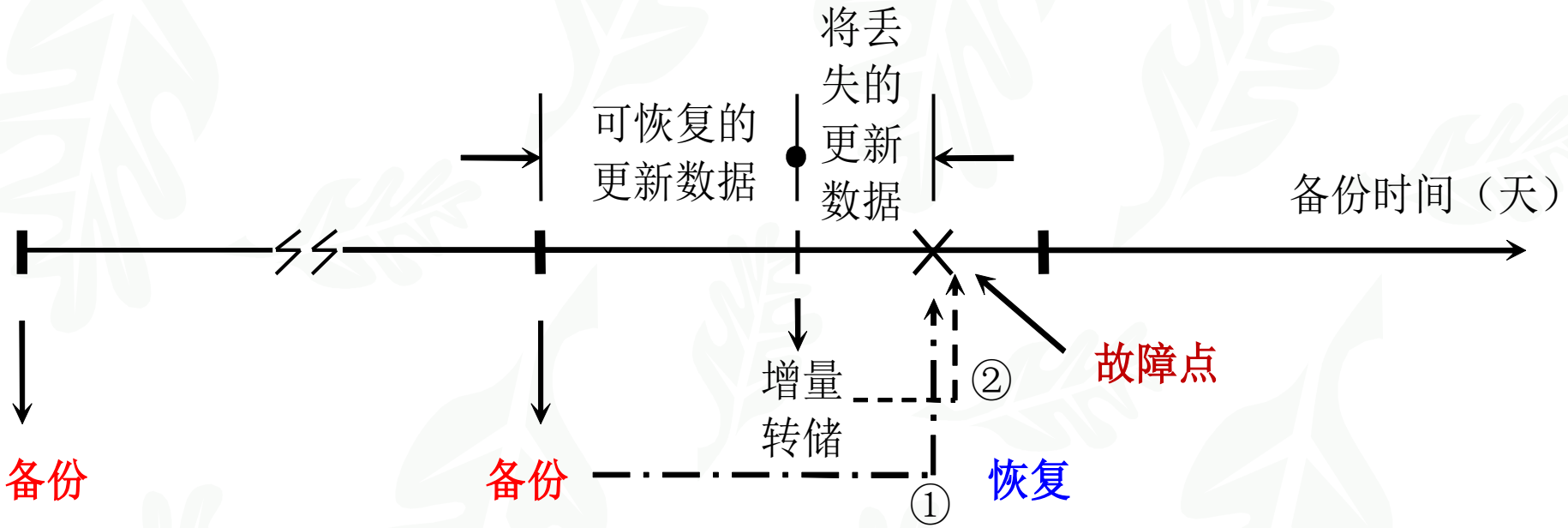


图 采用增量转储后的DB恢复

技术特点：优点是实现简单，缺点是不能恢复到尽可能近一点的一致状态。只适用于小型或不重要DBS。

二.以后备副本和日志为基础的恢复技术

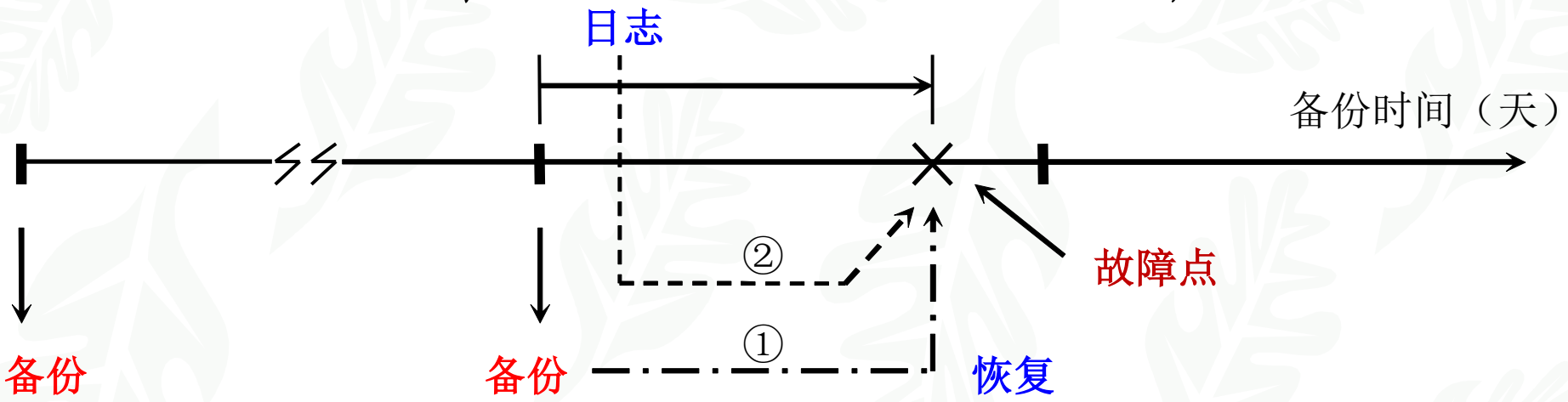


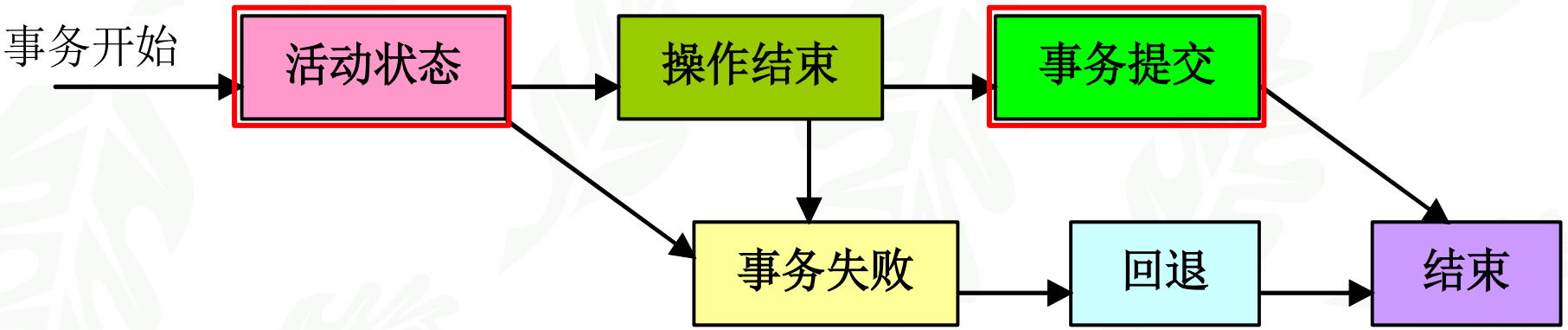
图 以后备副本和日志为基础的DB恢复

日志（Log）：供恢复用的DB运行情况的记录。其内容有：前像、后像和事务状态。

前像（BI）：事务更新DB所涉及物理块更新前的映象(Before Image, BI)。如撤销(Undo)更新，可利用BI使DB恢复到更新前的状态——前像回退。

后像（AI）：事务更新DB所涉及物理块更新后的映象(After Image, AI)。如重做(Redo)更新，可利用AI使DB恢复到更新后的状态——后像重做。

事务状态：根据事务状态变迁，事务状态有活动状态、操作结束、事务提交、事务结束、事务失败、回退。



事务状态变迁图

事务的两种结局：①提交而结束，其对DB的更新才能被其它事务访问；②事务失败，事务所作的DB更新必须回退。

故障事务处理的依据：事务是已提交，还是未提交。

系统处理：事务一旦开始，即记入日志，并跟踪事务两种状态，一是活动状态（以记录正在执行的事务），二是提交状态（以区分事务是否提交）。

恢复方法：故障排除后，先取最近副本，然后根据日志中的事务是否提交作相应处理。对未提交事务，一般应用前像（BI）回退（Undo）；对已提交事务一般用后像（AI）重做（Redo）。

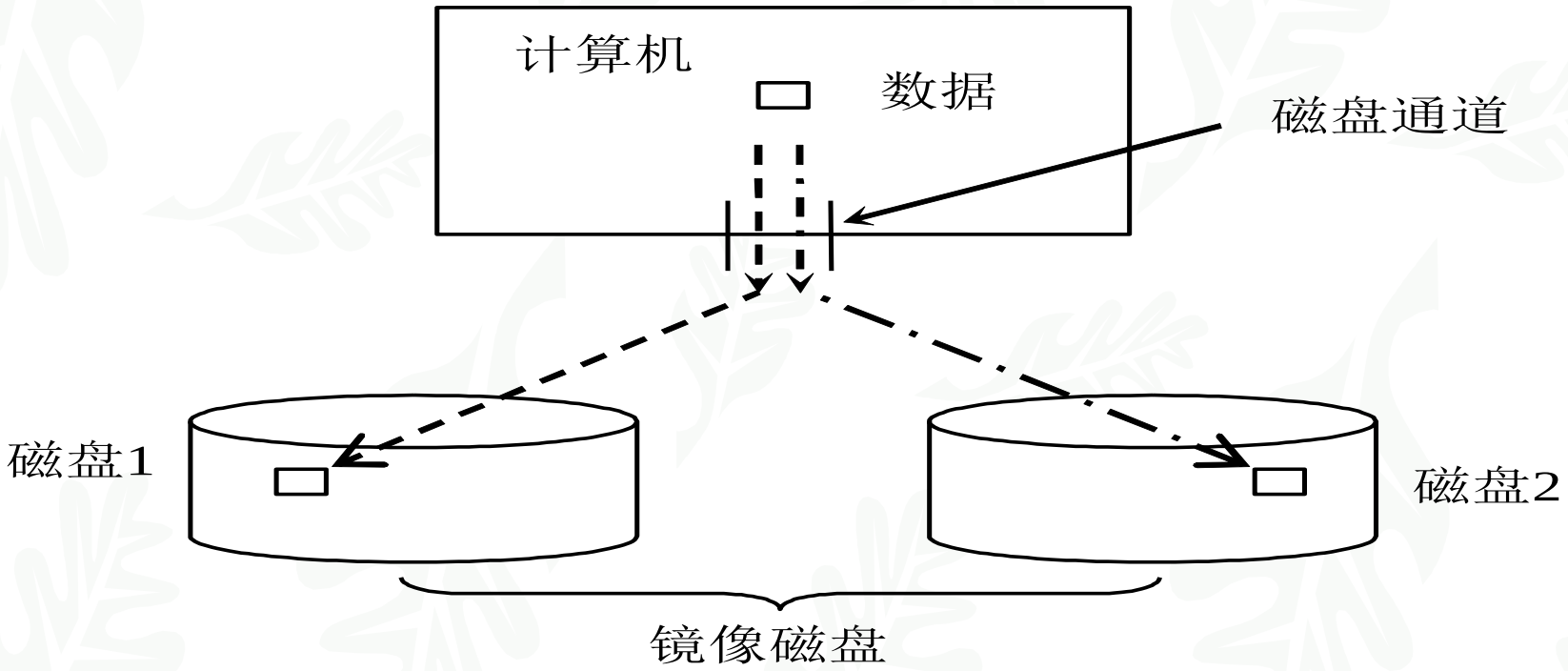
各种操作下AI、BI的内容和恢复方法

操作	前像 BI	后像 AI	Undo（撤销）	Redo（重做）
INSERT	无 / 空 / 不存在	新插入的完整记录	删除该记录	重新插入该记录
UPDATE	修改前的旧值	修改后的新值	用 BI 改回旧值	用 AI 写入新值
DELETE	被删除的完整记录	无 / 空 / 不存在	用 BI 恢复记录	删除该记录

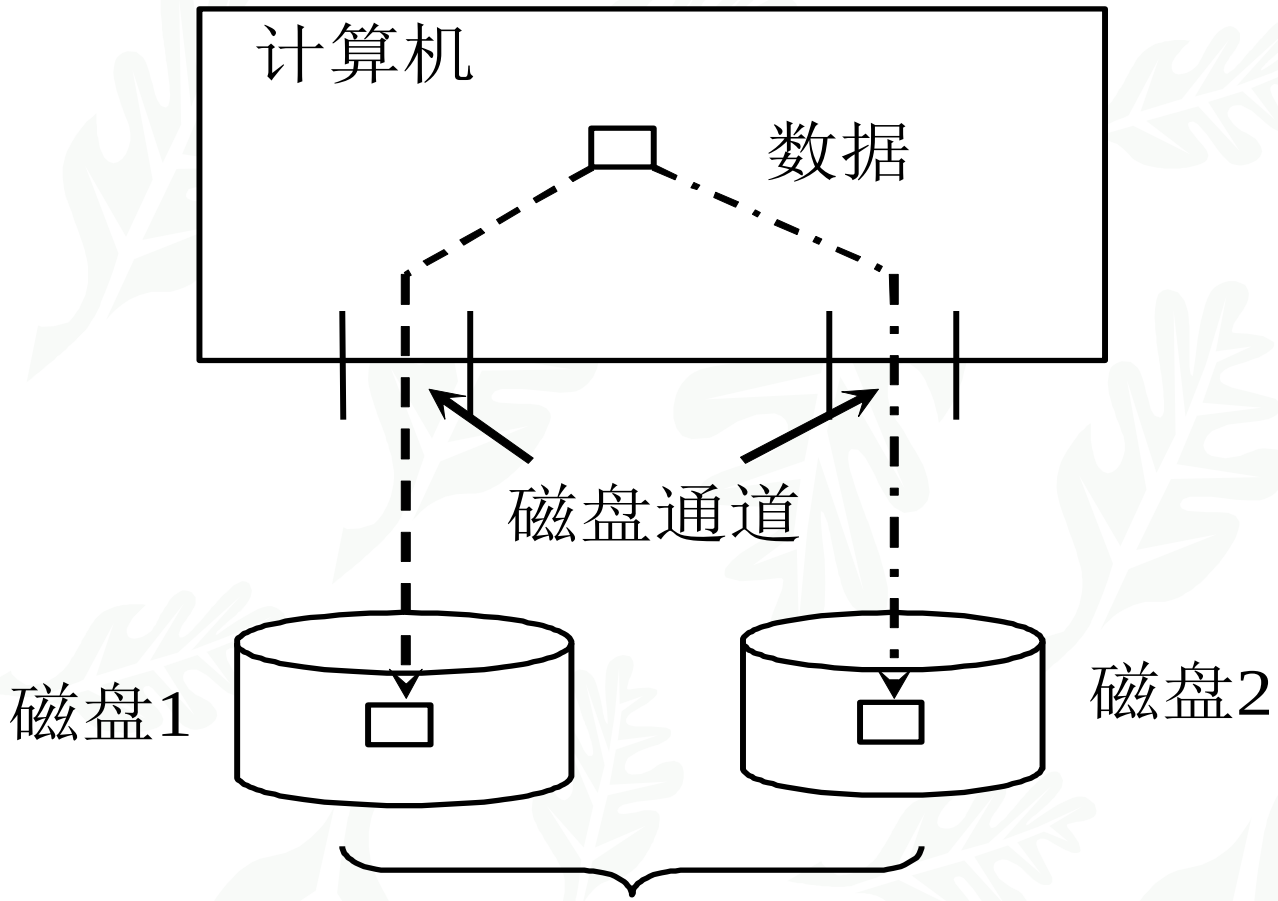
三.基于多副本的恢复技术

方法：利用多副本互为备份、恢复。

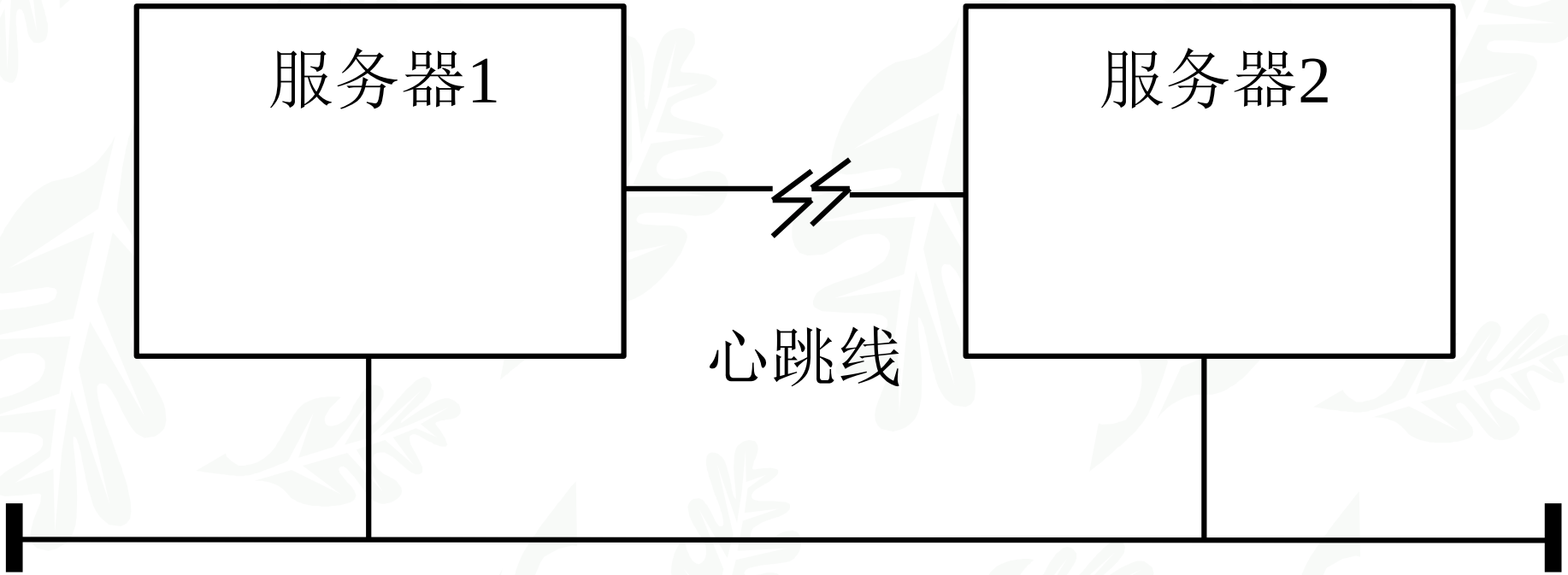
NOS/OS级的可靠性技术：镜像磁盘（Mirroring Disk）、双工磁盘（Duplex Disk）、镜像服务器（Mirroring Server）、群集（Cluster）系统。



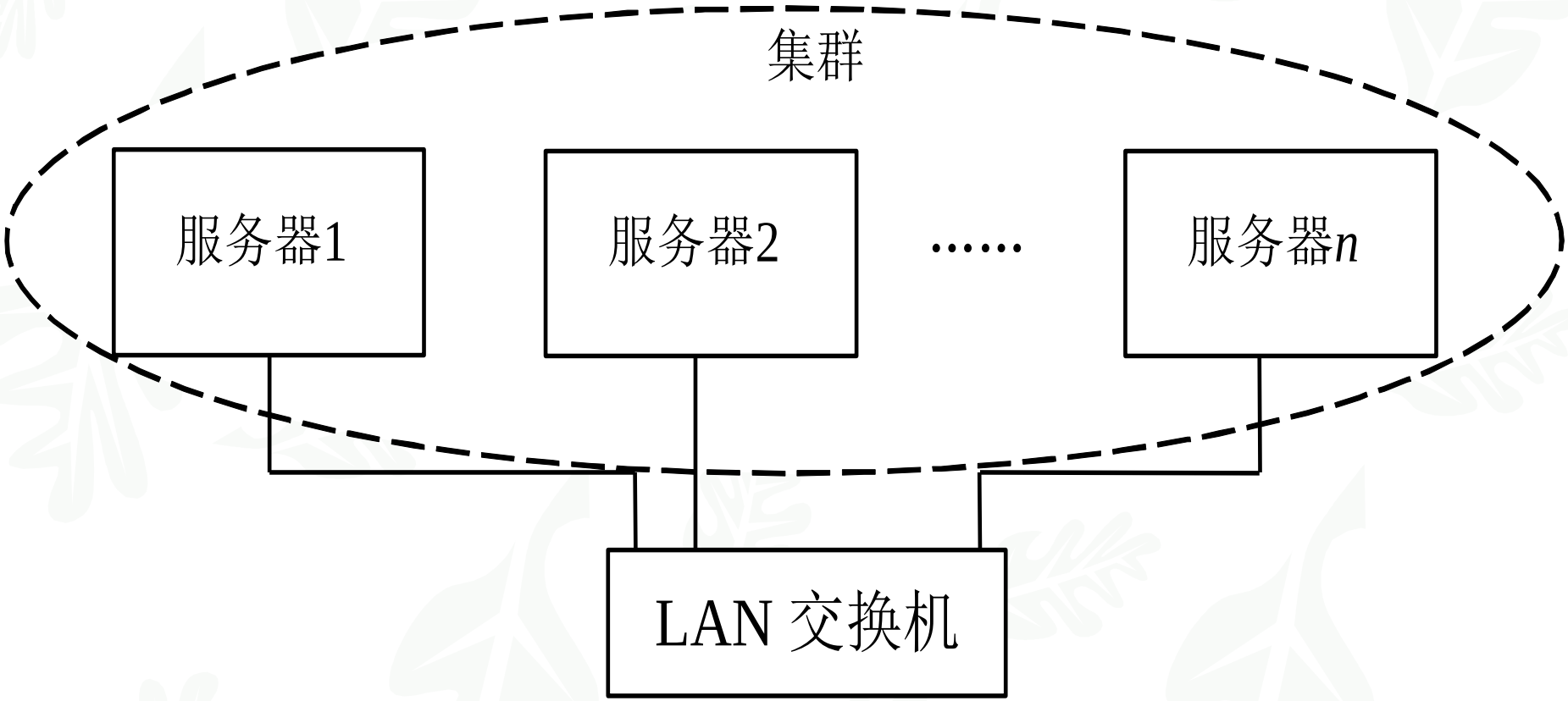
磁盘镜像



磁盘双工



服务器镜像



集群服务器

DBMS级的可靠性技术：数据库复制(Replication)、数据库镜像、日志镜像。

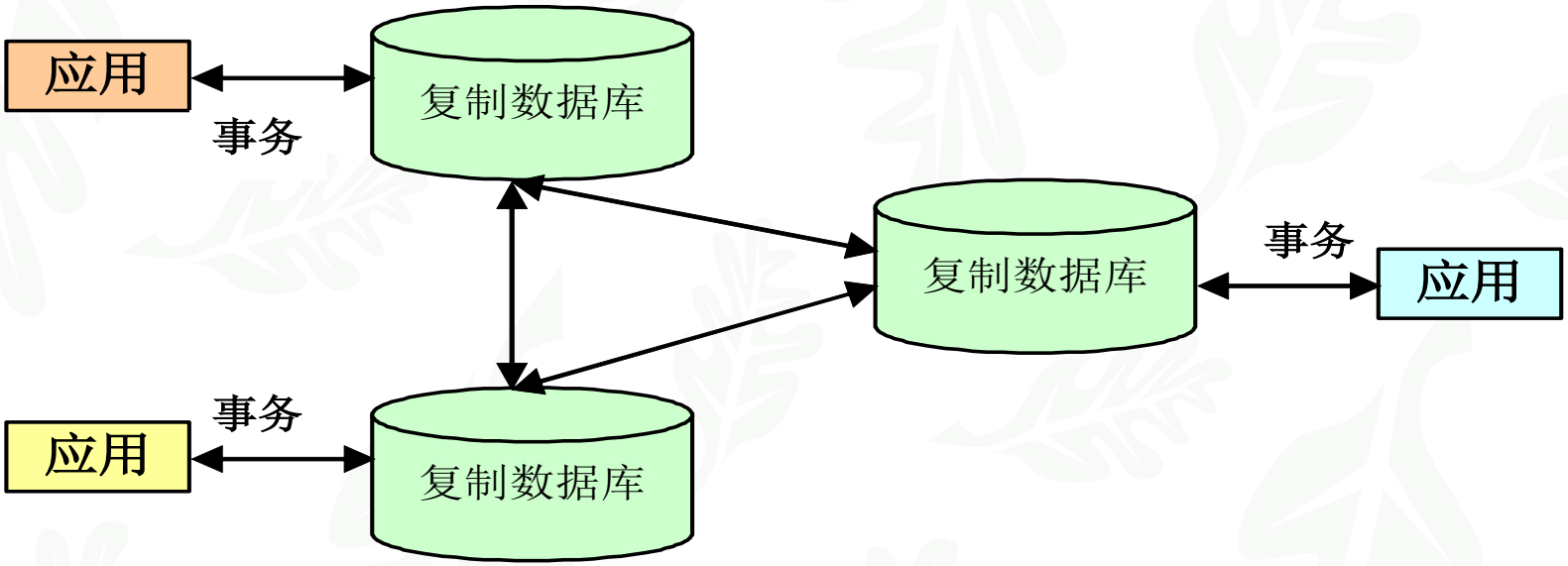
数据复制技术：在多个场地保留多个数据库副本，各个场地的用户可以并发地存取不同的数据库副本。通过分布式的复制服务器（Replication Server）实现。

数据复制要求：DBMS须采取一定手段用户对DB的修改能及时反映到所有副本上。

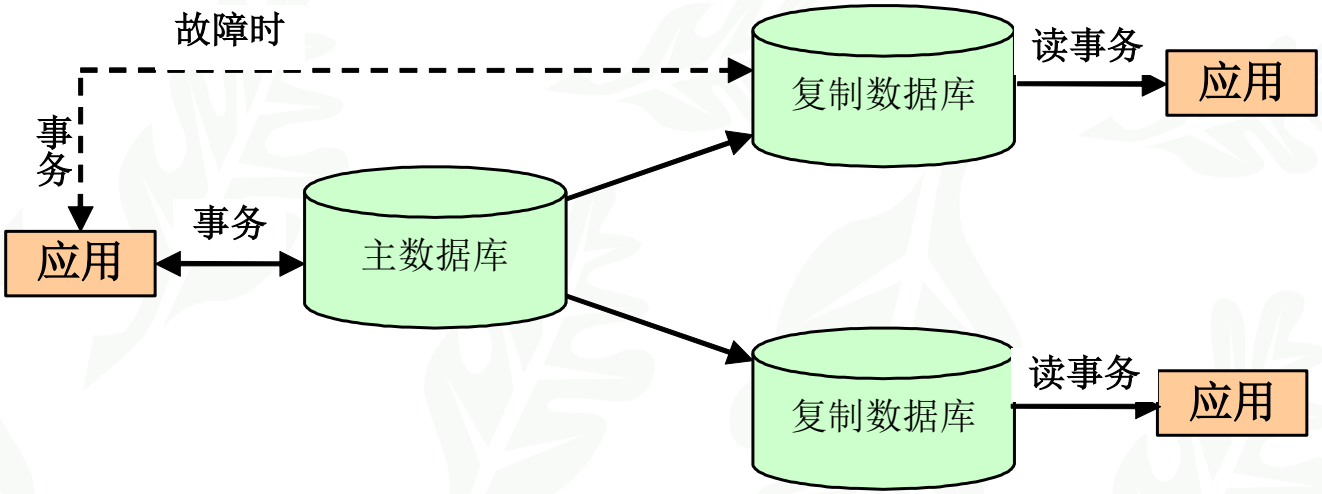
数据复制优点：(1) 当一个用户为修改数据加锁时，其他用户可访问数据库副本；(2) 当数据库出现故障时，系统可用副本进行联机恢复，恢复过程中，用户可继续访问副本而不中断应用。

数据库复制的三种方式：对等(peer-to-peer)复制、主从(master/slave)复制和级联(cascade)复制。

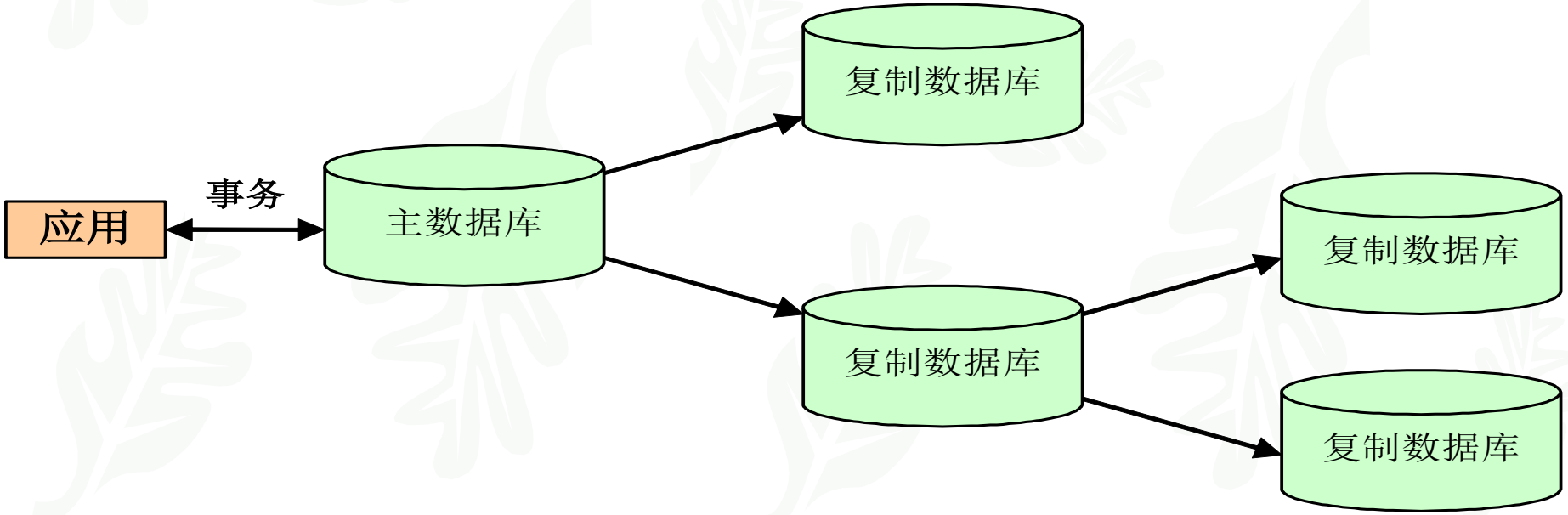
对等复制：是理想的复制方式。各场地DB地位平等，可互相复制数据。用户可在任一场地读取/更新公共数据，DBMS应将更新数据复制到所有副本。



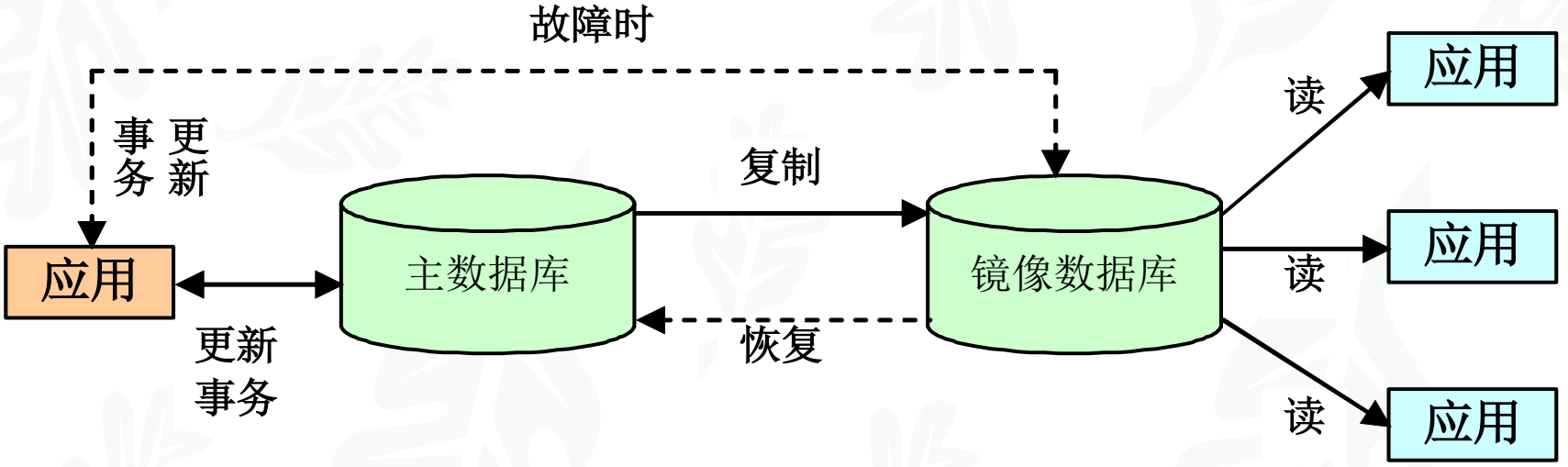
主从复制：数据只能从主DB复制到从DB。更新数据也只能在主场地进行，从场地供用户读数据。当主场地出现故障时，更新数据的应用可转到某一从场地。



级联复制：从主场地复制来的数据又从该场地复制到其他场地。级联复制可平衡当前各种数据需求对网络流量的压力。



数据库/日志镜像：DBMS为避免磁盘介质故障，提供日志与数据库镜像，由DBMS根据DBA要求，自动将整个DB或其中关键数据复制到另一个磁盘的数据库上。当主DB更新时，DBMS自动将更新后的数据复制到镜像数据库。出现介质故障时，可由镜像数据库提供应用服务，并利用它恢复主数据库。其实现通过数据复制进行。



7.3 日 志

日志(Log)记录： DB恢复所必需的数据。

日志及DB的存放： 日志丢失 → DB无法恢复 → 日志、DB分别存放(分区、磁盘、磁带副本)。

一.日志基本内容(不同DBMS略有差别)

① 活动事务表(Active Transaction List, ATL)：记录正在执行、尚未提交的事务标识符(Transaction Identifier, TID)。

② 提交事务表(Committed Transaction List, CTL)：记录已提交事务的标识符。

注意：提交时，先做提交的TID → CTL，后从ATL中删除该TID。

③ 前像(BI)文件

④ 后像(AI)文件

二.恢复后对日志的处理

① 不保留已提交事务的前像：已提交事务，只会做redo，不会做undo，故其前像可不保留；

② 有选择地保留后像：当更新内容写入DB后，如磁盘不出故障，后像可不保留。而磁盘故障机率小，故有些DBMS(如ORACLE)允许用户作出是否保留后像的选择；

③ 合并后像：对给定逻辑块号，只需保留最近的后像即可。



7.4 DBMS围绕更新事务的工作

回答如下问题:

1. DBMS需要对进行数据更新的事务做什么样的外围操作，才能在出现故障时顺利地恢复之?

2. 如何安排这些事务外围操作的执行步骤?

3. 安排事务外围操作步骤时应遵守的规则?



一.事务外围操作安排时应遵守的规则

1. 提交规则(Commit Rule)

该规则要求：后像应在提交前写入DB或写入日志(均在非易失存储器上)中。

说明：

① ACID持久性要求。

② 提交规则不要求后像一定在提交前写入DB，写入日志中亦可，即：如后像已写入日志，即使未写入DB或未完全写入DB，事务仍可提交，待事务提交后再继续写入DB。如此期间出现故障，可用日志的后像重做；其它事务可从缓冲区(未写入DB前更新内容仍在缓冲区)访问更新后的内容。

2. 先记后写规则(Log Ahead Rule，或WAL——write ahead logging)

该规则要求：如后像在事务提交前写入DB，需先将前像写入日志(非易失存储器上)，以便事务失败后，撤销事务所做的更新。

二. 事务状态及DBMS对应的工作安排

按后像写入DB时间及事务状态的不同，有三种安排方案。

方案1：后像在事务提交前完全写入DB

(1) DBMS工作步骤安排

事务执行状态	DBMS应做的工作
开始状态	步骤1：TID → ATL /* 将事务TID记入日志的ATL */
事务内的操作执行状态	无（事务内的增、删、改操作在内存中进行）
提交前	步骤2：BI → Log /* 将BI写入日志，先记后写规则的要求 */
	步骤3：AI → DB /* 将AI写入DB，提交前后像完全写入DB，本方案的要求 */
提交状态	步骤4：TID → CTL /* 提交更新事务 */
提交后	步骤5：从ATL删除TID

(2) 故障时的事务恢复措施

如果事务按上步骤执行过程中发生故障，可根据下表进行恢复。

ATL	CTL	事务所处状态	恢复措施
有	无 (未提交)	步骤1已完成，但步骤4尚未完成	① 若BI已➔Log,则undo；否则勿需undo; ② 从ALT删除TID。
有	有 (已提交)	步骤4已完成	从ALT删除TID
无	有 (已提交)	步骤5已完成	勿需处理

※注：① “有”与“无”,表示事务标识符TID是否在ATL或CTL表中。

② CTL表中的“有”与“无”,表示事务“已提交”与“未提交”。

方案2：后像在事务提交后才写入DB

(1) DBMS工作步骤安排

事务执行状态	DBMS应做的工作
开始状态	步骤1：TID → ATL /* 将事务的TID记入日志的ATL */
事务内的操作执行状态	无（事务内的增、删、改操作在内存中进行）
提交前	步骤2：AI → Log /* 遵循提交规则的要求 */
提交状态	步骤3：TID → CTL /* 提交更新事务 */
提交后	步骤4：AI → DB /* 本方案的要求 */
	步骤5：从ATL删除TID

(2) 故障时的事务恢复措施

如果事务按上步骤执行过程中发生故障，可根据下表进行恢复。

ATL	CTL	事务所处状态	恢复措施
有	无	步骤1已完成，但步骤3未完成	从ALT删除TID。
有	有	步骤3已完成，步骤5未完成	①redo； ②从ALT删除TID。
无	有	步骤5已完成	勿需处理

方案3：后像在事务提交前后写入DB

(1) DBMS工作步骤安排

事务执行状态	DBMS应做的工作
开始状态	步骤1：TID → ATL /* 将事务的TID记入日志的ATL */
事务内的操作执行状态	无（事务内的增、删、改操作在内存中进行）
提交前	步骤2：AI，BI → Log /* 遵循提交规则和先记后写规则的要求 */
	步骤3：AI → DB /* 部分AI写入DB，本方案要求 */
提交状态	步骤4：TID → CTL /* 提交更新事务 */
提交后	步骤5：AI → DB /* 将剩余AI写入DB，本方案要求*/
	步骤6：从ATL删除TID

(2) 故障时的事务恢复措施

如果事务按上步骤执行过程中发生故障，可根据下表进行恢复。

ATL	CTL	事务所处状态	恢复措施
有	无	步骤1已完成，但步骤4尚未完成	① 若BI已➔Log,则undo；否则勿需undo； ② 从ALT删除TID。
有	有	步骤4已完成，但步骤6未完成	①用AI进行redo； ②从ALT删除TID。
无	有	步骤6已完成	勿需处理

7.5 事务内消息的处理

一.问题提出

消息及处理要求：事务一般会给用户发送消息，告之其执行情况或要求用户做某些动作。发送消息也是事务执行结果的一部分，亦应遵循Nothing or All原则。但消息与数据不同，在很多情况下很难用undo来消除其影响。

消息管理器及其作用：一般，要求在事务结束(提交或回退)前，不能发送消息。这样，发送消息就不是事务的任务，必须委托第三方执行，一般委托给系统的消息管理器(Message Manager, MM)完成。

二.消息的具体处理

(1) 方法

① 事务执行时：将消息送给MM，MM为每个事务建立一个消息队列；

② 事务中止时：MM在恢复时将该事务的消息丢弃，以消除其影响；

③ 事务正常结束时：事务通知MM发送消息；MM将消息队列存入不易失存储器中；MM允许事务在正常结束前增加或删除消息。

(2) MM的发送处理

方法：MM采用“发送—确认”方式传送。消息发出后，如确认超时，则再发送一次，直到收到确认信号为止。为防止用户端重复收到同一消息，对消息编号，当用户端收到同一消息时不予理睬。

注意：消息仅指事务中发送给用户的消息，不包括事务执行过程中，由系统发给用户的消息，如语法错等。

7.6 失效类型及恢复对策

故障恢复能力不是无限的，它只针对某些概率较高类型的失效。

一.失效类型

① 事务失效（恢复时无需人工干预，只用undo操作）

② 系统失效（恢复时无需人工干预，undo、redo都可能用到）

③ 介质失效（恢复时需要人工干预，只用redo操作）

二.事务失效及恢复

(1) 基本概念

事务的成功执行与结束： 一个事务以BEGIN TRANSACTION语句的成功执行开始，以COMMIT或ROLLBACK语句的成功执行结束。

提交点： COMMIT建立了一个**提交点**(Commit Point)，在商用DBMS产品中也叫**同步点**(Syncpoint)。

提交点含义： 提交点标志着逻辑工作单元的结束，要求DB处于一致状态。

(2) 事务失效的原因

① 事务无法执行而中止；

② 用户主动撤销事务；

③ 因系统调度差错而中止。

(3) 事务失效的恢复措施

① MM丢弃该事务的消息队列；

② 如需要，进行undo操作；

③ 从ATL删除该事务TID，释放该事务所占资源。

三.系统失效及恢复

(1) 基本概念

系统失效包括：

① 掉电；

② 除DB存储介质故障之外的软硬件故障。

(2) 系统失效的恢复措施

① 重新启动OS和DBMS；

② 恢复DB到一致状态（未提交事务进行undo，对已提交事务进行redo）。

(3) 系统失效的恢复措施的改进

问题提出：故障时，未提交的事务在系统中不多，故恢复时undo的工作量不大；已提交的事务则很多，因不能肯定其后像是否已写入DB，只得做redo，而redo相当费时。

检查点及其目的：为减少redo工作量，DBMS一般每隔一定时间在日志中设置一个检查点(Check Point, CP)。

检查点工作：在检查点，DBMS强制写入所有已提交事务的后像。这样，在检查点以前提交的事务，恢复时勿需redo。

检查点的间隔参数：可以由DBA在DBMS初始化时设置，并可在运行时调整。

(4)取CP过程

① 暂停事务执行；

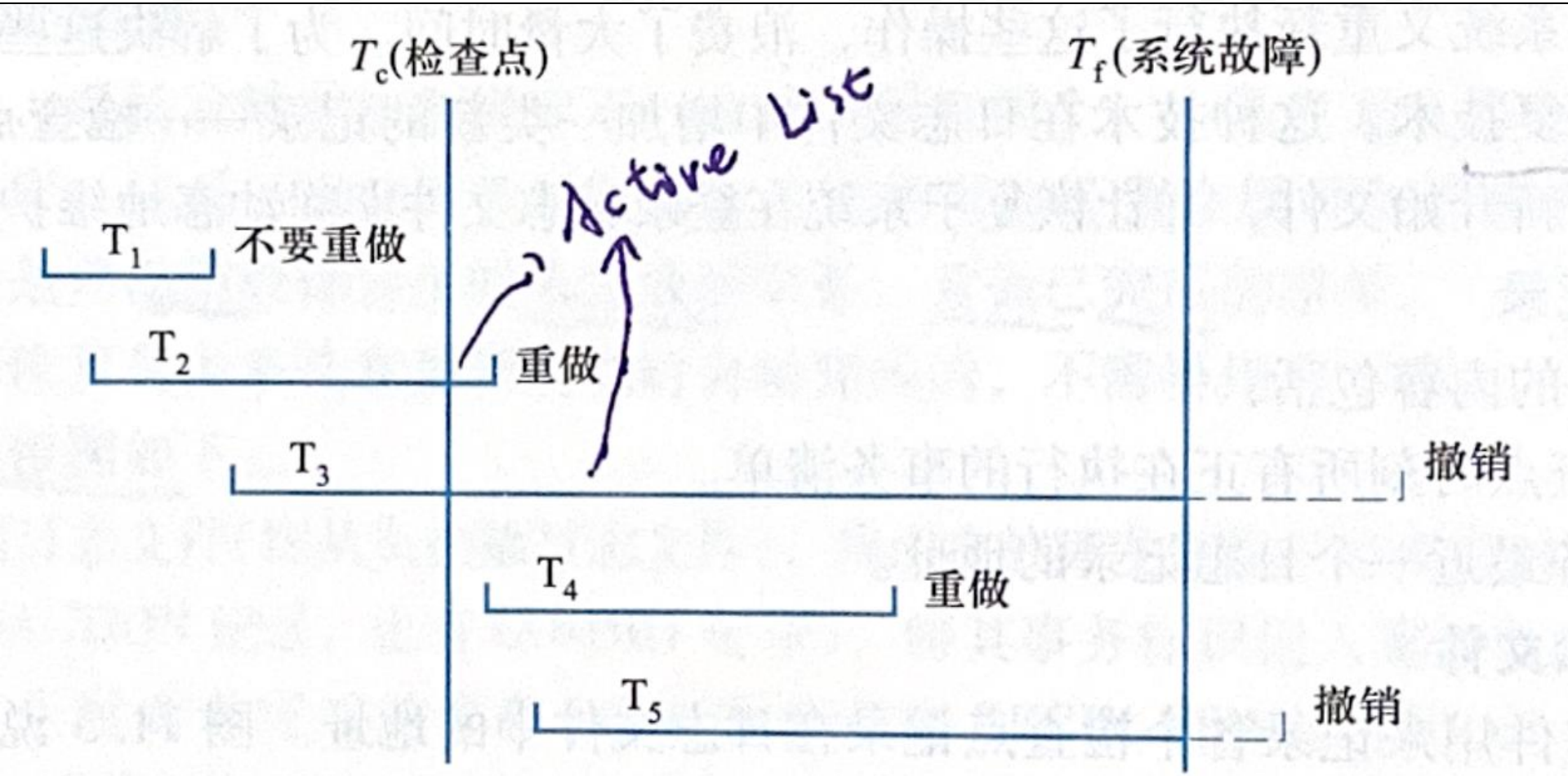
② 向磁盘中写入上一个CP以后所提交事务留在内存中的后像；

③ 在日志的提交事务表（CTL）中记下检查点；

④ 恢复事务的执行。

注意：取CP对DBMS正常运行影响较大，且只有在发生系统失效时才能发挥其减少redo工作量的作用。因此，在DBMS忙碌时少设/暂停设置CP。

带检查点的系统失效及恢复策略
（引自《数据库系统概论》（第6版））



三.介质失效及恢复

(1) 基本概念

介质失效：指磁盘发生故障的DB受损。一般，磁盘失效机率很小。

(2) 介质失效的恢复措施

① 修复系统，必要时更换磁盘；

② 如系统（OS或DBMS）崩溃，重新启动系统；

③ 加载最近后备副本；

④ 用日志中的后像，redo取最近后备副本以后提交的所有事务。

一个补充例子（引自《数据库系统概论》（第6版））：

4. 考虑表 11.2 所示的日志记录：

① 如果系统故障发生在序号 14 之后，说明哪些事务需要重做，哪些事务需要回滚。

② 如果系统故障发生在序号 10 之后，说明哪些事务需要重做，哪些事务需要回滚。

③ 如果系统故障发生在序号 9 之后，说明哪些事务需要重做，哪些事务需要回滚。

④ 如果系统故障发生在序号 7 之后，说明哪些事务需要重做，哪些事务需要回滚。

5. 考虑表 11.2 所示的日志记录，假设开始时 A、B、C 的值都是 0：

① 如果系统故障发生在序号 14 之后，写出系统恢复后 A、B、C 的值。

② 如果系统故障发生在序号 12 之后，写出系统恢复后 A、B、C 的值。

③ 如果系统故障发生在序号 10 之后，写出系统恢复后 A、B、C 的值。

④ 如果系统故障发生在序号 9 之后，写出系统恢复后 A、B、C 的值。

⑤ 如果系统故障发生在序号 7 之后，写出系统恢复后 A、B、

表 11.2　习题 11.4 日志记录示例

序号	日志
1	T_1：开始
2	T_1：写 A，$A=10$
3	T_2：开始
4	T_2：写 B，$B=9$
5	T_1：写 C，$C=11$
6	T_1：提交
7	T_2：写 C，$C=13$
8	T_3：开始
9	T_3：写 A，$A=8$
10	T_2：回滚
11	T_3：写 B，$B=7$
12	T_4：开始
13	T_3：提交
14	T_4：写 C，$C=12$

补充例子参考答案：

答：

① 重做：T_1、T_3；回滚：T_2、T_4。

因为 T_1 和 T_3 事务已经提交，所以重做；T_2 回滚，T_4 执行中，没有提交，所以要回滚。

② 重做：T_1；回滚：T_2、T_3。

因为 T_1 事务已经提交，所以重做；T_2 回滚，T_3 执行中，没有提交，所以要回滚。T_4 事务还没有开始。

③ 重做：T_1；回滚：T_2、T_3。

④ 重做：T_1；回滚：T_2。

答：

① A=8，B=7，C=11。

② A=10，B=0，C=11。

③ A=10，B=0，C=11。

④ A=10，B=0，C=11。

⑤ A=10，B=0，C=11。

⑥ A=0，B=0，C=0。（全部回滚到初值）